Agentic Engineering in Practice

Jamie Wang
Developer Advocate

Hemanth Battu
Ecosystem Engineer

IBM

Our roadmap:

Module 0: Get to Know Your Tool

Module 1: Requirements Gathering and Spec Creation

Module 2: Implementation

Module 3: Verification

Module 4: Balancing Cost Through Efficiency

Module 5: Justifying an AI Workflow

link to lab guide

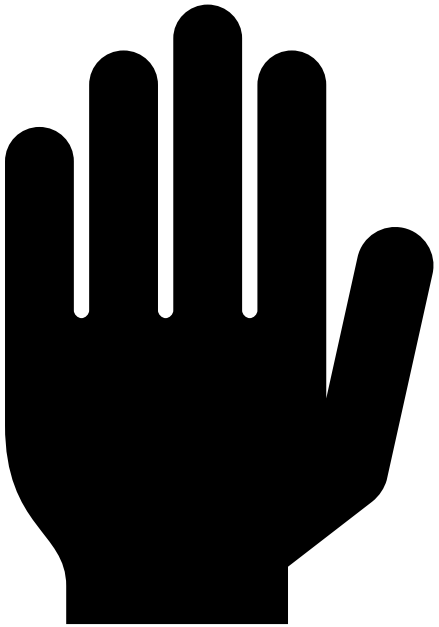
https://ibm.biz/txc-agentic-eng

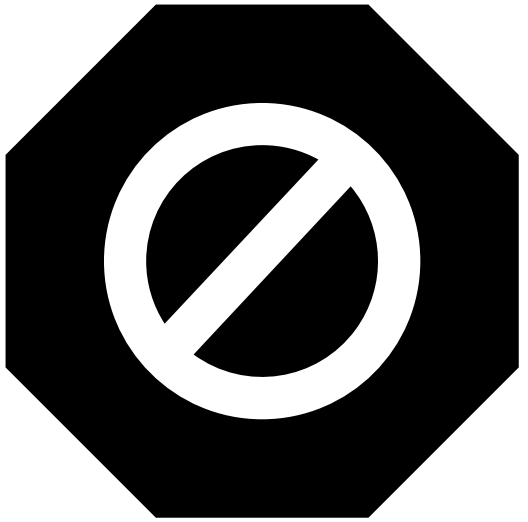
Follow the Prework
to start the lab

Introduction to Agentic Engineering

STOP CODING SLOP

PLEASE





There are three main vectors by which to judge your agentic engineering process:

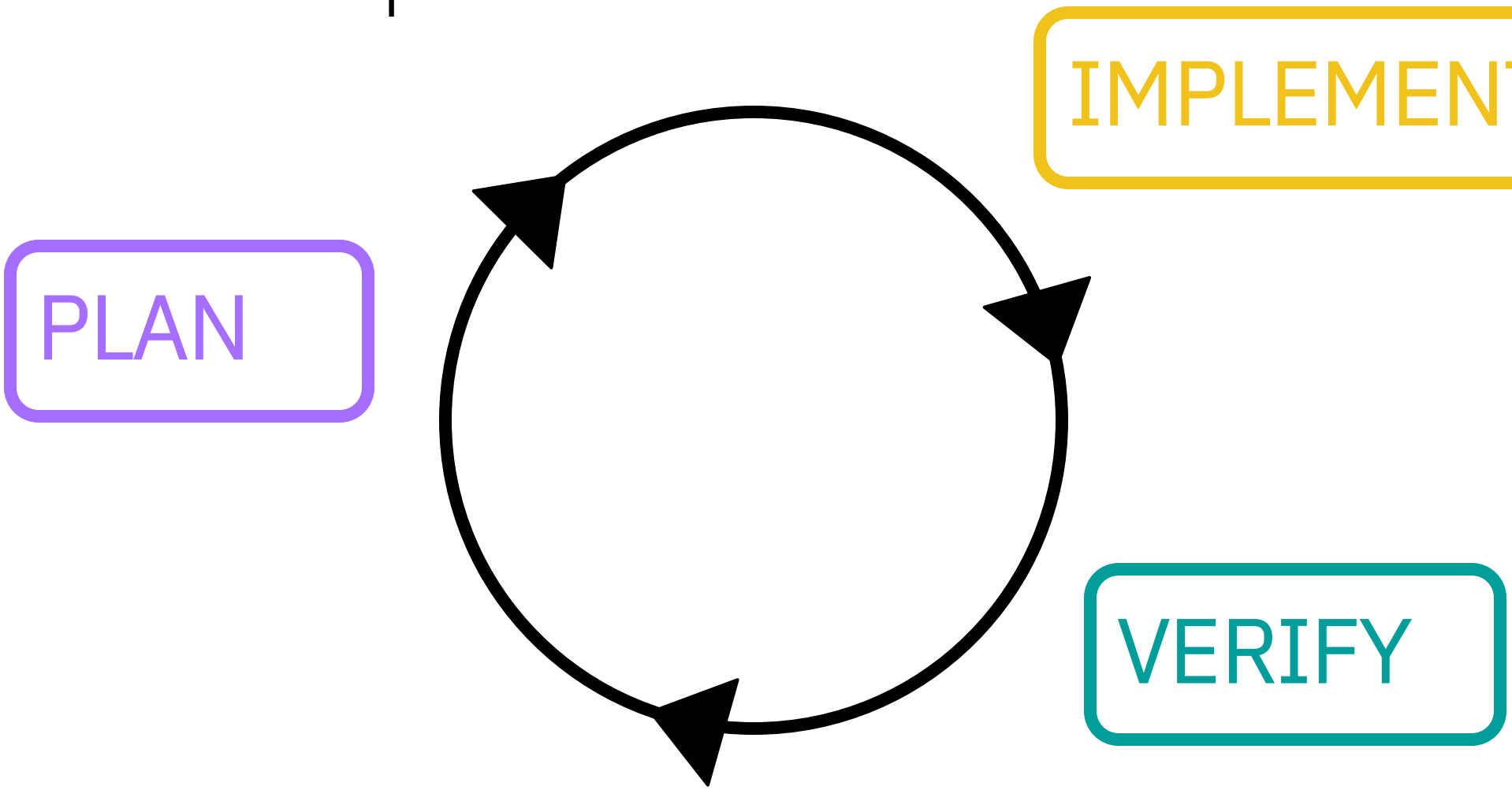
PROJECT TRUST: The trustworthiness of the project being worked on

HUMAN OWNERSHIP: Your ability to control the state of the project

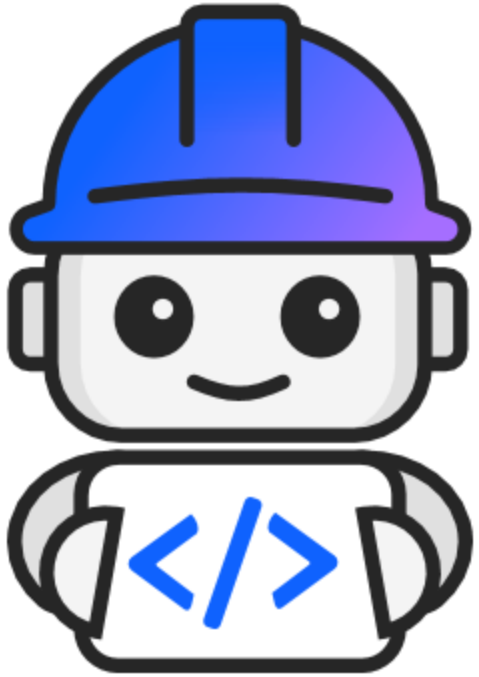
AGENT AUTONOMY: The agent's ability to make independent progress

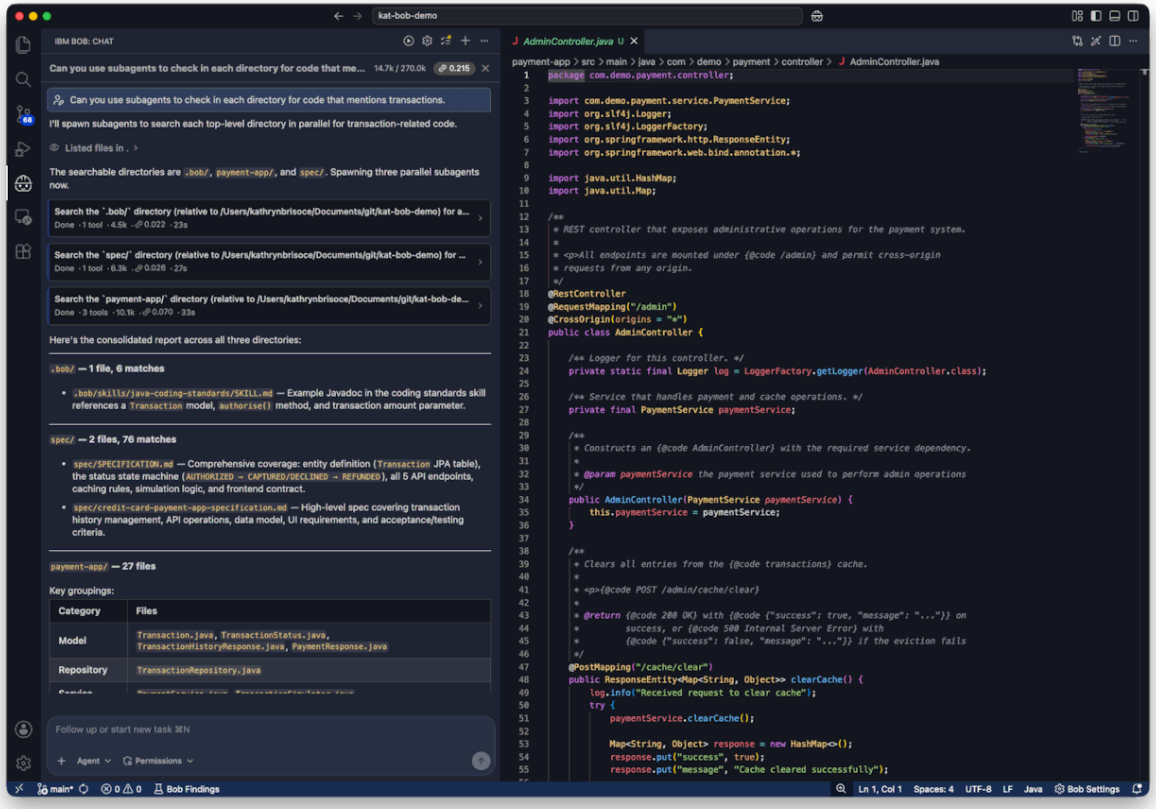
Your goal:
Maximize autonomy without sacrificing trust and ownership

Basic Workflow of Spec Driven Development



Module 0: Get to Know Your Tool

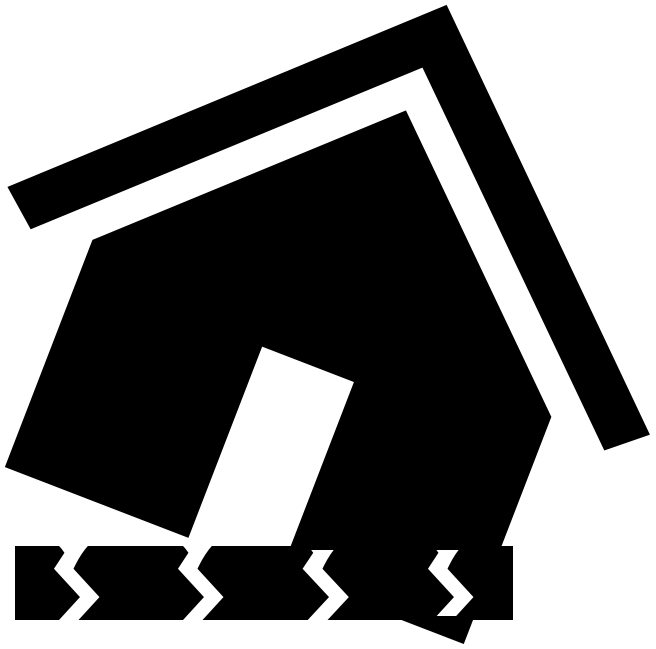


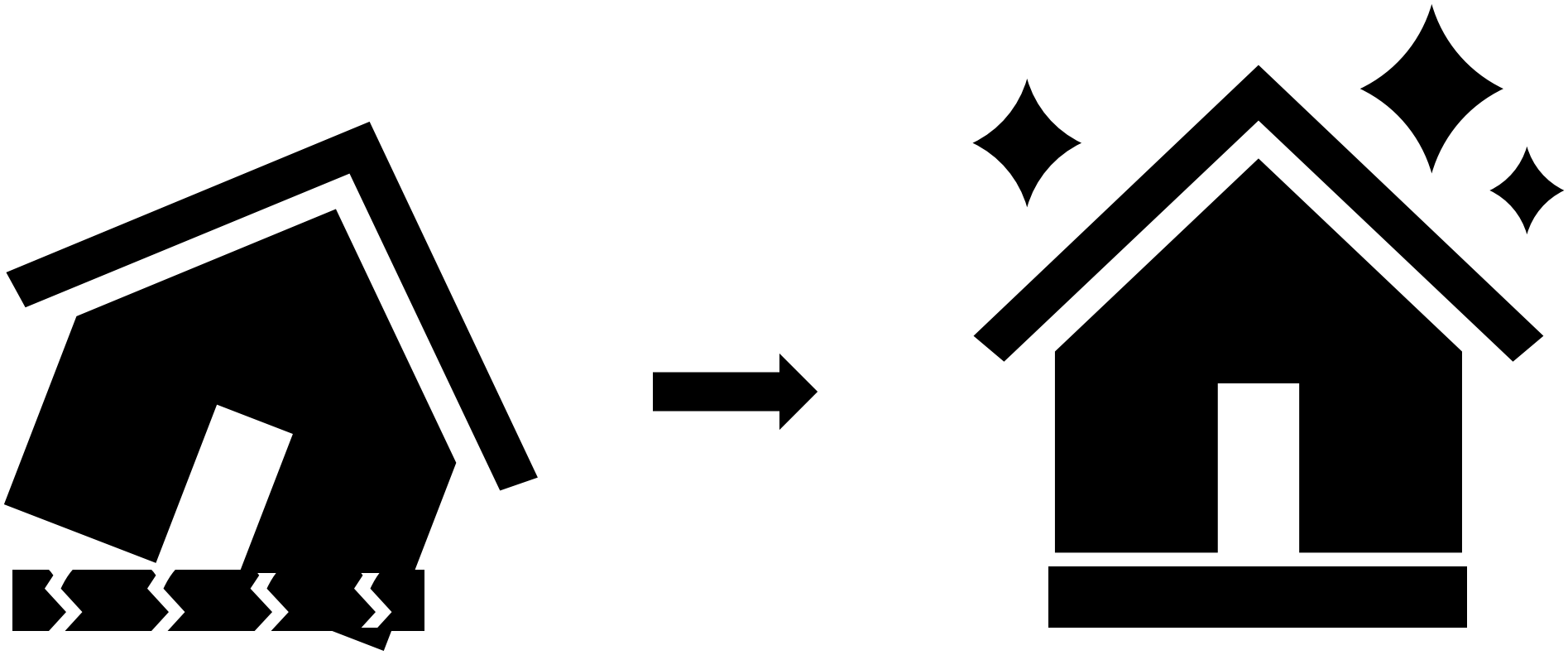


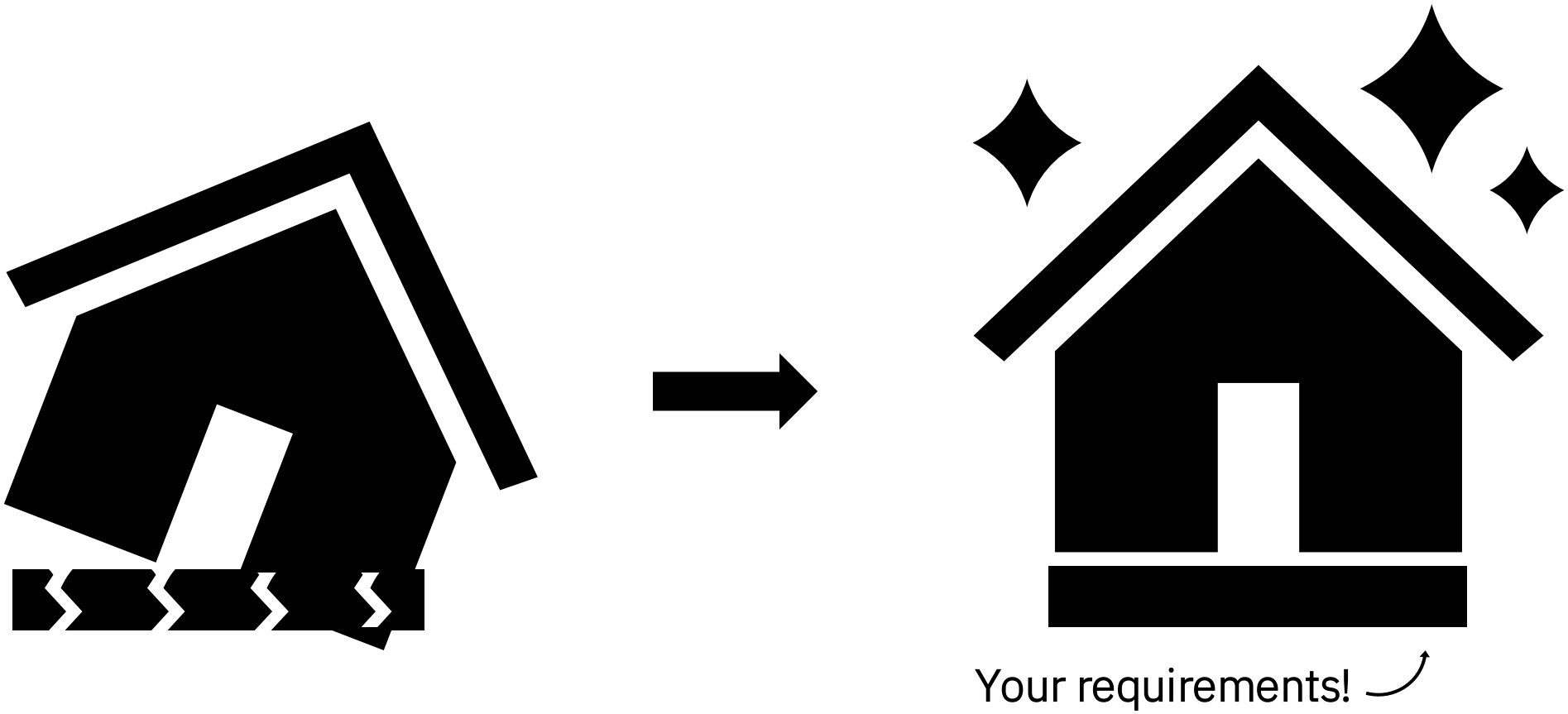
IBM Bob IDE Walkthrough

Module 1: Requirements Gathering and Spec Creation








Your requirements!

Requirements should be…

- unambiguous
- testable
- evidence based

“The account list page must display the first page of results (up to 50 accounts) within 2 seconds.”



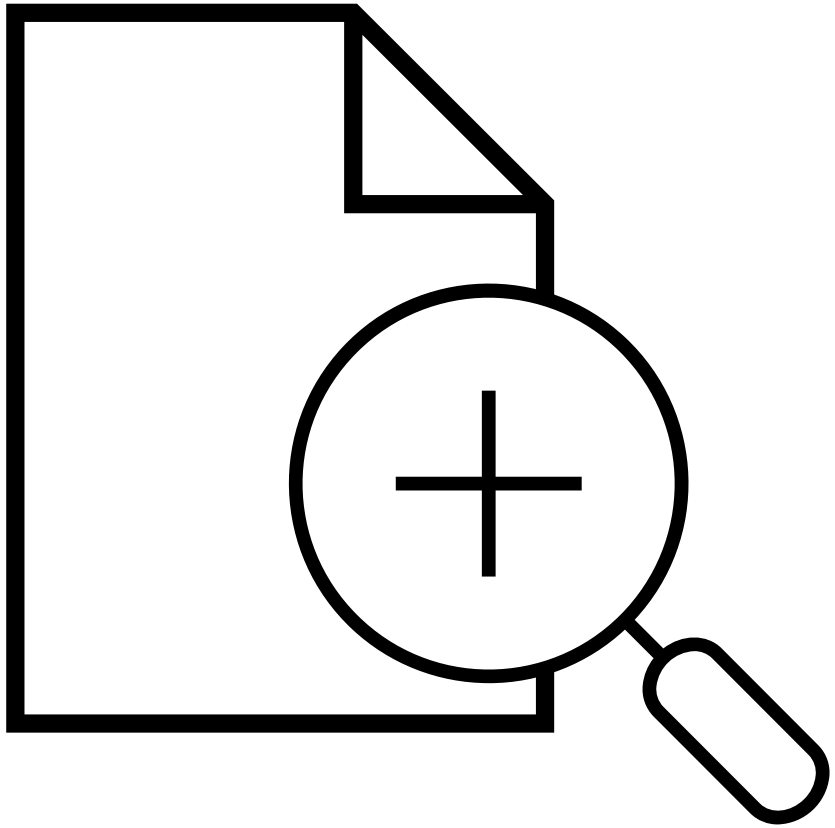
Validation Criteria



The Initial Prompt

"Using my requirements and notes, draft a phased implementation plan. Don't write code yet. The spec should include..."

Reviewing and Refining



You are responsible for the code you ship, and you are responsible for the way you ship it.

Balancing autonomy and ownership takes judgement.

Introducing Your Codebase

Module 1 Walkthrough

Module 2: Implementation

Supervision is a balancing act



Agentic Engineering Progression:

- Prompt Engineering
- Context Engineering
- Loop Engineering
- Graph Engineering

let’s talk terminology…

Prompt Engineering

- Optimizing your input to tailor the output of your LLM
- Exists in a single conversation between you and the agent
- Relies on the specificity and clarity of each individual message

Context Engineering

- Curating relevant information presented to the model via context
- Manages context-window noise and cost
- Foundational to further forms of agentic engineering

Loop Engineering

- Building a system by which the agent iterates and loops through different tasks
- Agent prompts itself and decides what to do next from your spec
- Though much more autonomous, is still a linear process

Graph Engineering

- Developing an agent graph to coordinate work between multiple agents
- Each agent performs a designated step within an iteration
- Forms a multi-agent pipeline
- Enables parallelization

Agentic Implementation

Prompt Engineering

- Optimizing your input to tailor the output of your LLM
- Exists in a single conversation between you and the agent
- Relies on the specificity and clarity of each individual message

Context Engineering

- Curating relevant information presented to the model via context
- Manages context-window noise and cost
- Foundational to further forms of agentic engineering

Loop Engineering

- Building a system by which the agent iterates and loops through different tasks
- Agent prompts itself and decides what to do next from your spec
- Though much more autonomous, is still a linear process

Graph Engineering

- Developing an agent graph to coordinate work between multiple agents
- Each agent performs a designated step within an iteration
- Forms a multi-agent pipeline
- Enables parallelization

The amount of autonomy you can give your agent comes from your ability to design a system that allows it to succeed.

Module 2 Walkthrough

Module 3: Verification

Common failure modes:

- Structural *Implementation failure*

  Missing functionality *Incomplete abstractions* *Pseudocode*

- Content *The code exists, but should it?*

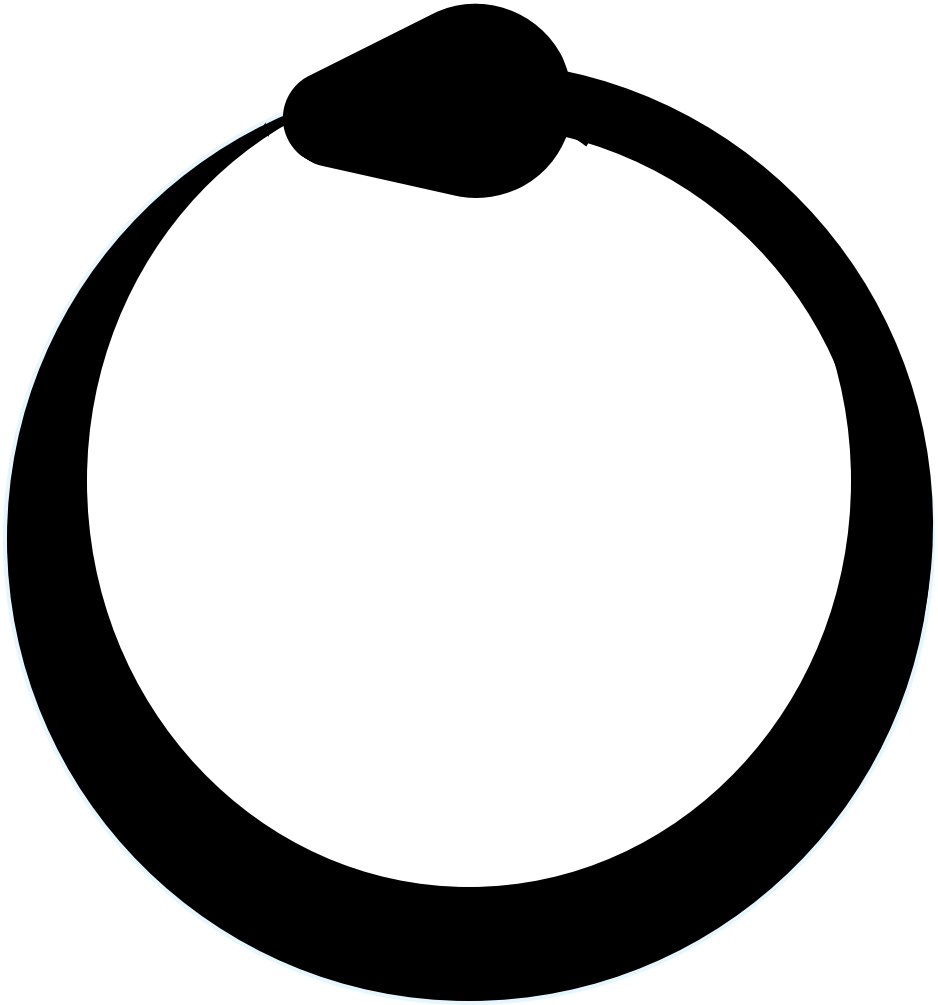
  Local vs. Global Issues *Missing edge cases* *Incorrect dependency*

- Stylistic *Everything works, but you're not proud of it*

  Dead code *Hardcoded values* *Outdated libraries*

Review — human or AI?

AI accelerates work, but it also **amplifies risk**.



Make sure the trust you put in your agent is proportional to the trust you expect others to place in you.

Module 3 Walkthrough

Module 4: Balancing Cost Through Efficiency

Getting more done yay!

means...

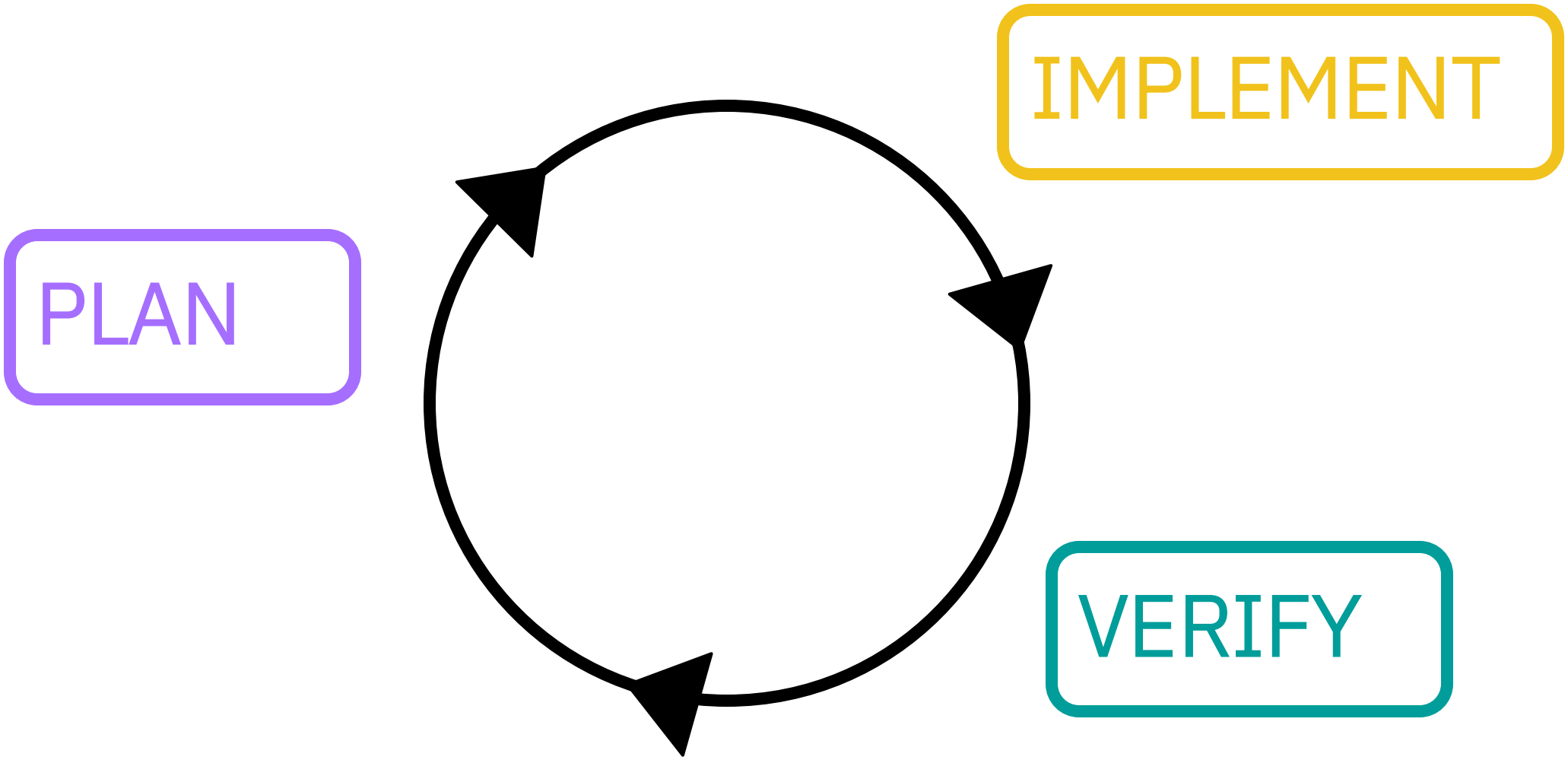
Using more resources hmmm...

means...

Driving up costs boo :(

it’s normal... but what can we do about it?

wow, it all comes back around!



Plan: Data Collection Through Benchmarking

What is driving up cost?

- Excessive output tokens?
- Mass code ingestion?
- The model itself?

Implement: Cost Saving Methodologies

Potential solutions

- The ’caveman’ skill
- Offloading tasks to local models
- Model routing

Verify: Rerun Against Your Benchmarking Criteria

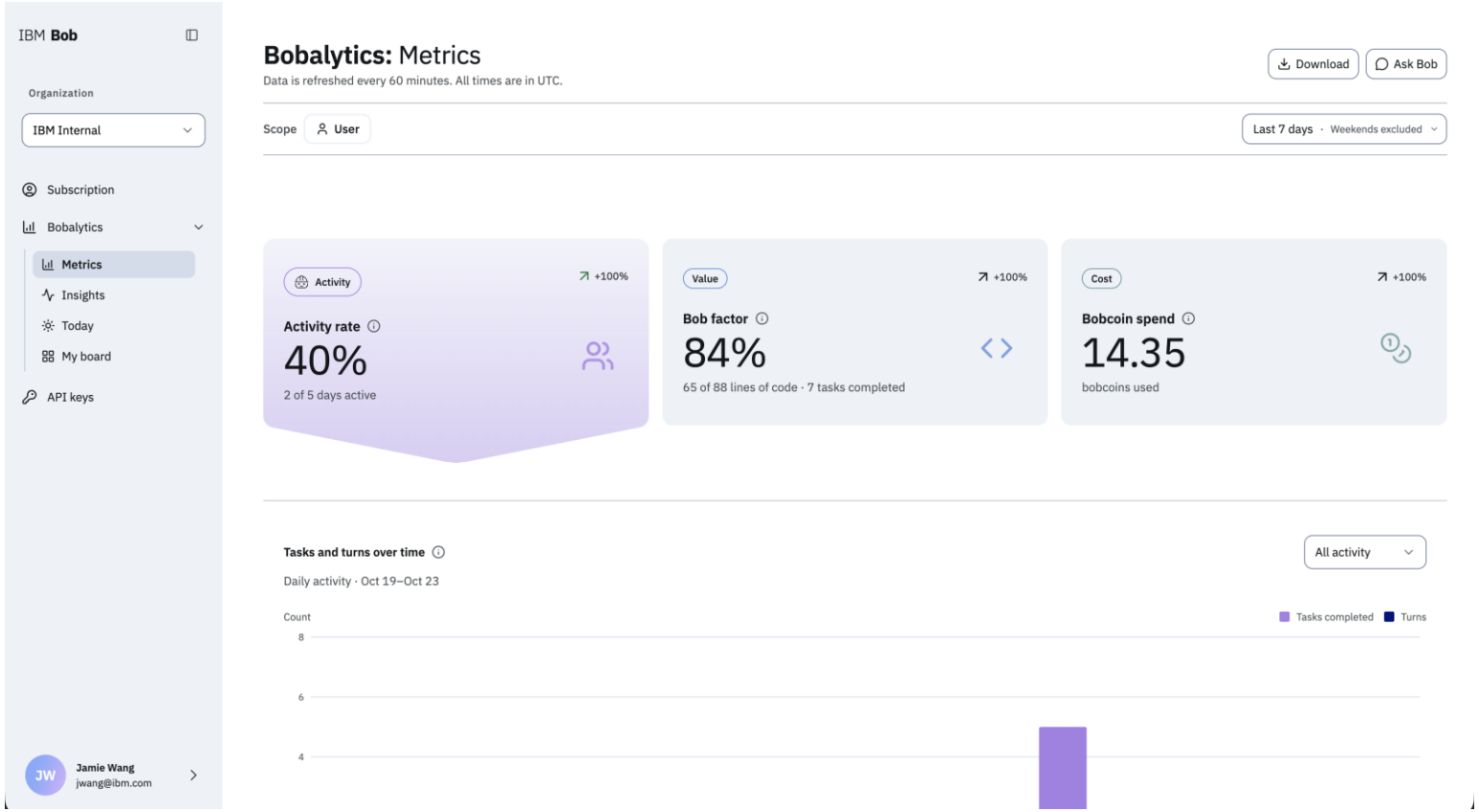
Understanding the problem



Measuring the solution

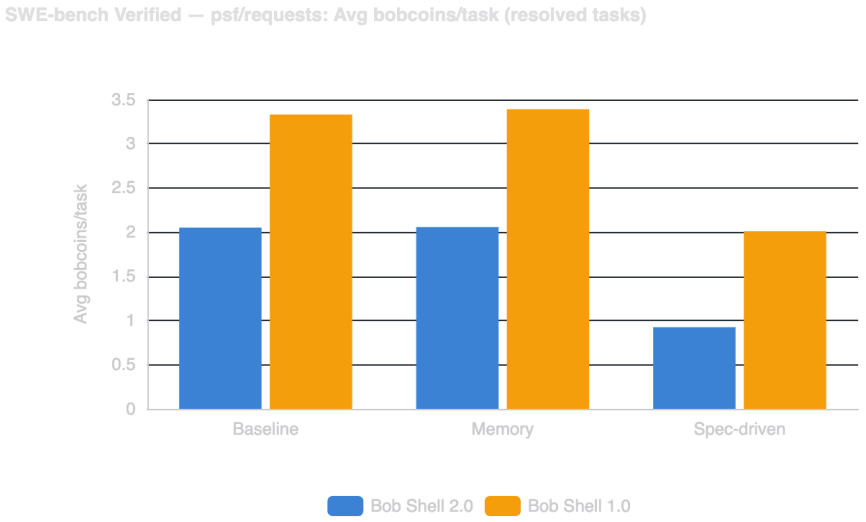
Verify: Rerun Against Your Benchmarking Criteria

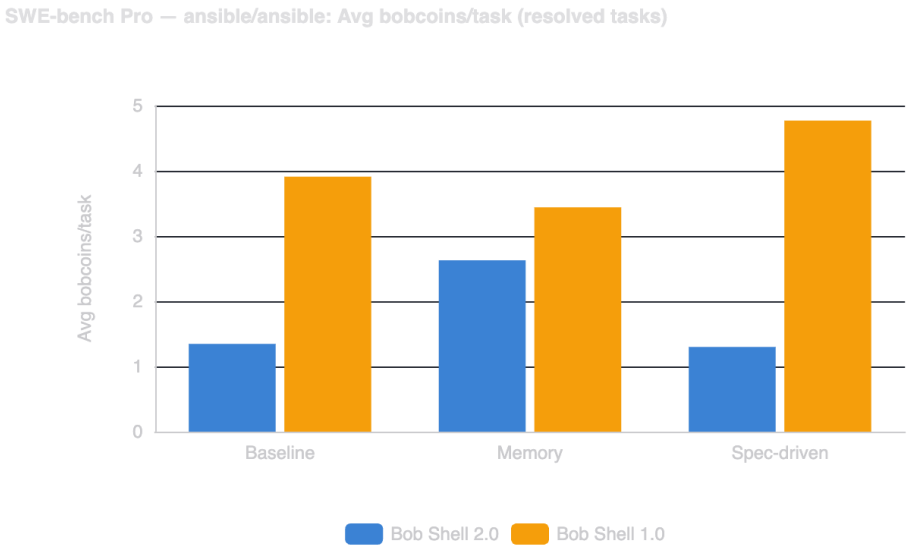
Bobalytics shows your activity rate, daily usage, and task breakdown



Verify: Rerun Against Your Benchmarking Criteria

SWE-bench *https://www.swebench.com/*





Module 4 Walkthrough

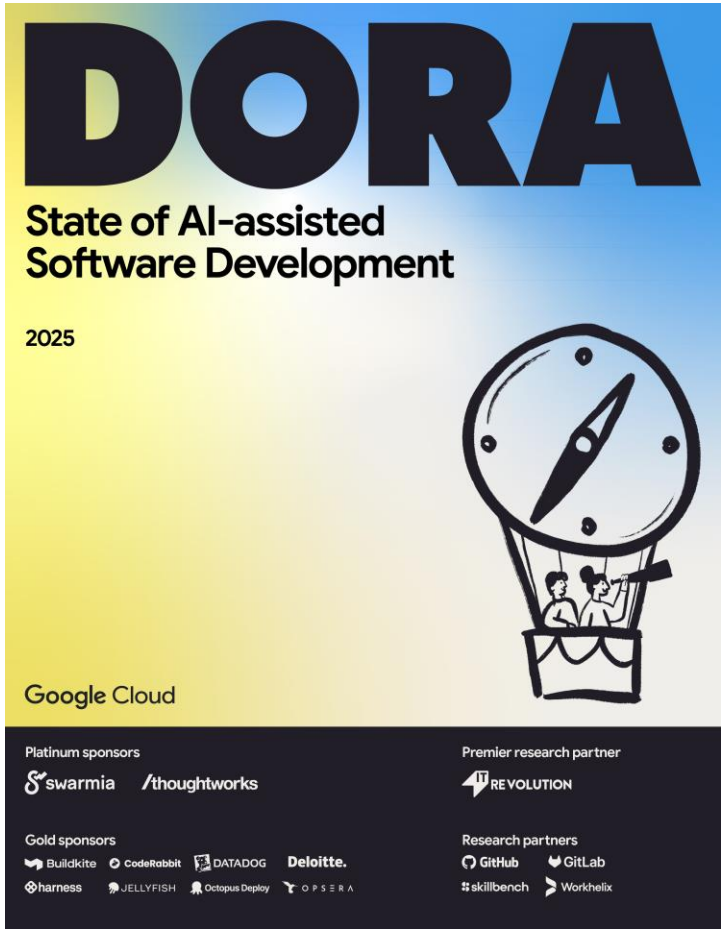
Module 5: Justifying an AI Workflow

Software delivery performance metrics have always been necessary, now more so than ever.

It measure not what you're doing, but **why**.

DORA's key output metrics are divided into two categories:

Throughput and **Instability**.



dora.dev/dora-report-2025

Throughput measures how much ***more*** you can deliver.

- Change lead time
- Deployment frequency
- Failed deployment recovery time

Instability measures how much ***better*** you can deliver.

- Change failure rate
- Deployment rework rate

Goodhart's Law

"When a measure becomes a target, it ceases to be a good measure."

IBM **SkillsBuild**

Build the AI development skills employers are looking for.

Learn through hands-on experiences with IBM Bob:

- AI-assisted software development
- Real-world development workflows
- Portfolio-ready projects
- Industry-recognized credentials

Scan to explore
IBM Bob in action ↷



